

TECHNICAL DOCUMENTATION

nanoGPT

From Scratch to Served: Pre-training, RLAIFF Alignment,
and Deployment of Two Small Language Models

Models covered: Fire (33M parameters) · Fiko (15M parameters)

Pipeline: Tokenization → Pre-training → Synthetic Data Generation → SFT → DPO →
Serving

Live deployment: nanogpt-ic7p.onrender.com

Repository: github.com/yogeshsikhwal77/nanoGPT

License: MIT · © 2026 Yogesh Sikhwal

Preface

This document consolidates the complete technical documentation for **nanoGPT**, a self-contained project that takes two small decoder-only language models — Fire (33M parameters) and Fiko (15M parameters) — through every stage of a modern LLM lifecycle: tokenizer training, base pre-training, synthetic instruction-data generation, supervised fine-tuning, and two rounds of Direct Preference Optimization (DPO), before serving the result behind a production FastAPI backend.

What makes the project notable is not scale but discipline. Every stage — including the generation and grading of its own training data — runs on a single consumer GPU with no paid API calls and no human labeling. A locally-hosted 3-billion-parameter judge model (Qwen 2.5, served through Ollama) plays the role that a hosted frontier model or human annotators would typically play in a larger RLHF/RLAIF pipeline: it writes question-answer pairs from seed story data, and later grades pairs of candidate answers to build the preference data DPO trains on.

This merged edition brings together three previously separate documents into a single reference:

- **Part I — Architecture & Pipeline** covers the model design, a fully recorded end-to-end training run, the RLAIF data pipeline, and the project's file layout.
- **Part II — Training Guide** is the exact, verified command-line sequence for reproducing every stage, along with a configuration reference and the rough edges you'll hit if you follow the code (not just the happy path) as it currently exists.
- **Part III — Serving & Reference** documents the deployed FastAPI service, its API surface, hardware/memory footprint, known troubleshooting cases, current limitations, and the project roadmap.

Throughout, this documentation favors precision over polish where the two are in tension: several sections call out places where the shipped code does not quite match its own documentation or UI (for example, the chat interface offers six model tiers, but only two are ever actually loaded in production). These notes are left in deliberately, since they are exactly the details a new contributor or operator would otherwise lose time rediscovering.

Part I — Architecture & Pipeline

Contents

- Overview
- Why This Project Matters
- Architecture
- A Recorded Training Run
- The RLAIIF Pipeline
- Project Structure

Overview

Task	Answer factual questions about short children's stories, resistant to hallucination
Data	TinyStories for pre-training; TinyStories-Instruct as seed material for synthetic Q&A + preference pairs
Models	Fire (~33M params, 512 ctx) and Fiko (~15M params, 256 ctx) decoder-only Transformers
Tiers per model	Medium (SFT) → High (DPO v1) → Max (targeted DPO v2)
Data generation & judging	Local Qwen 2.5 (3B) via Ollama — writes SFT Q&A pairs and grades DPO preference pairs
Serving	FastAPI backend (<code>app/main.py</code>) + a static HTML/CSS/JS chat UI, deployed via Docker
Live deployment	<code>nanogpt-ic7p.onrender.com</code>
Offline	Training and data generation run fully locally, zero third-party API dependencies

Why This Project Matters

- **Automated synthetic dataset creation.** A local LLM (Qwen 2.5 3B via Ollama) both writes the SFT question/answer data and judges preference pairs, with zero paid API calls and zero human labeling.
- **Small models can be taught not to hallucinate.** Pre-training alone leaves a 15–33M parameter model prone to fabricating details or losing track of chronology. A subsequent DPO pass, driven by a local judge, measurably corrects these failure modes without touching model size.
- **Runs entirely on a consumer laptop GPU.** The full pipeline — tokenization, pre-training, synthetic data generation, SFT, DPO, and serving — is tuned to fit inside 6 GB of VRAM.
- **Production inference doesn't even need a GPU.** The deployed app's `requirements.txt` installs a CPU-only build of PyTorch (`--extra-index-url .../whl/cpu`); only training needs CUDA.

Architecture

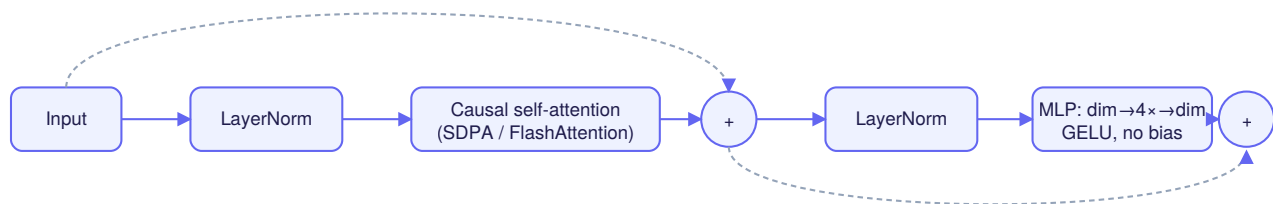
Both models share the same block design (`src/nanogpt/model.py`) and differ only in width, depth, and context length:

Hyperparameter	Fiko	Fire	Notes
Layers	6	8	Pre-norm decoder blocks
Attention heads	6	8	Head dim fixed at 64
Hidden size (<code>dim</code>)	384	512	Residual stream width
MLP intermediate	1,536	2,048	4× expansion, GELU activation, no bias
Context window	256	512	Tokens per sequence
Vocabulary size	10,000	10,000	Byte-level BPE (<code>tokenizer.json</code>)
Position embedding	Learned, absolute	Learned, absolute	<code>nn.Embedding(context_len, dim)</code> added to token embeddings
Precision	bfloat16 autocast	bfloat16 autocast	Native Ampere/Ada Lovelace tensor cores
Attention kernel	SDPA	SDPA	<code>F.scaled_dot_product_attention(..., is_causal=True)</code> (FlashAttention)
Weight tying	Yes	Yes	<code>lm_head.weight = tok_emb.weight</code>

You can get the exact parameter count for any config with:

```
python -m nanogpt.model # instantiates GPT from configs/base.yaml and prints model.num_params()
```

Decoder Block



Pre-norm decoder block (`src/nanogpt/model.py`) — identical design for Fire and Fiko; dashed lines are residual (skip) connections

A Recorded Training Run

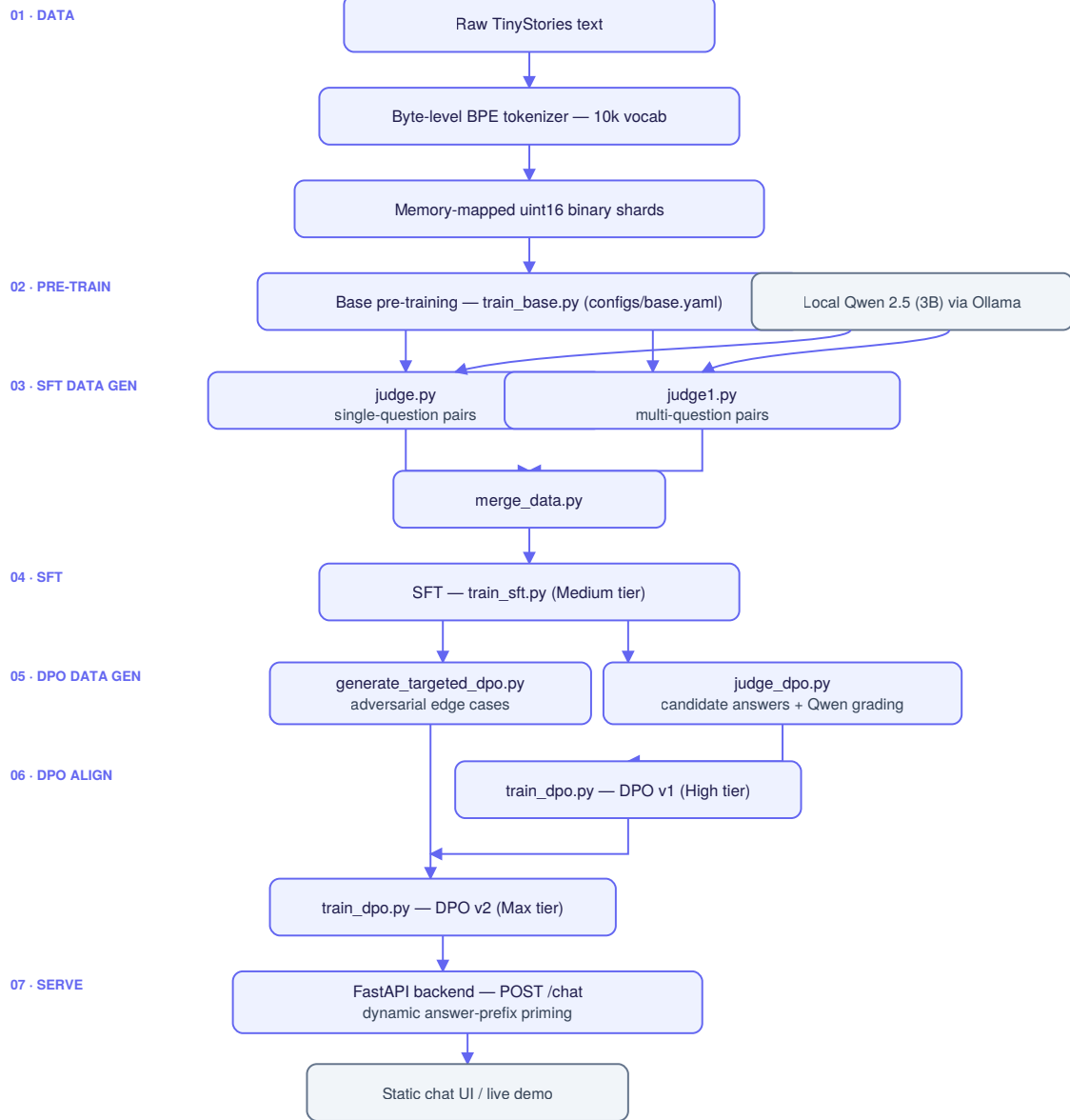
Numbers below are from an actual run of the Fire (33M) pipeline, end to end:

Stage	Result
Tokenizer training	10,000-vocab BPE tokenizer trained on ~276 MB of raw text
Base pre-training	7,424 steps, val_loss 1.4520 , 71.4 minutes
SFT data generation	1,000 single-question pairs (~9 min) + 1,458 multi-question pairs from 500 stories (~15 min)
SFT (2 epochs)	Epoch 1 val_loss 1.1364 → Epoch 2 val_loss 1.0540 (best), new checkpoint saved each time
DPO preference generation	500 candidate pairs attempted → 292 graded pairs saved (judge skipped/discarded the rest)
DPO v1 (baseline)	DPO loss stayed close to $\ln(2) \approx 0.693$ throughout — expected at the very start of preference training
Targeted failure mining	198 chronology/list-extraction contrastive pairs mined from the v1 checkpoint
DPO v2 (targeted, gentler lr/beta)	DPO loss dropped further, to ~0.667 by the end of the pass — a clearer, measurable preference signal than v1

Worth checking if you rerun this: in the DPO-preference-generation log, the judge's chosen answer ("Winner") was **B** for the large majority of the run shown. That's consistent with either a genuine quality difference between candidate generation strategies, or a positional/order bias in the judge prompt (`judge_dpo.py`) — worth a quick manual spot-check of a sample of "B" wins if you want to rule out order bias before trusting the resulting preference data.

No equivalent recorded run exists yet for Fiko (15M) in this repository.

The RLAIIF Pipeline



Stage	Script(s)	Typical output
1. Tokenizer	<code>tokenizers.py</code>	<code>tokenizer.json</code>
2. Base pre-training	<code>train_base.py</code>	<code>checkpoints/base_model_{15M,33M}.pt</code>
3. Hybrid SFT data gen	<code>judge.py</code> , <code>judge1.py</code> , <code>merge_data.py</code>	<code>data/sft/merged_pairs.json</code>
4. SFT	<code>train_sft.py</code>	<code>checkpoints/sft_model_{15M,33M}.pt</code>
5. DPO data gen	<code>generate_targeted_dpo.py</code> , <code>judge_dpo.py</code>	<code>data/dpo/dpo_pairs.json</code> , <code>data/dpo/targeted_dpo_pairs.json</code>
6. DPO alignment	<code>train_dpo.py</code> (run twice)	<code>checkpoints/dpo_model_{15M,33M}_v1.pt</code> → <code>..._v2.pt</code>
7. Evaluation	<code>evaluate.py</code>	Validation loss / perplexity
8. Serving	<code>app/main.py</code> , <code>app/inference.py</code>	<code>POST /chat</code>

Exact, verified commands for each stage are in **Part II — Training Guide**.

Project Structure

```

nanoGPT/
├── app/
│   ├── main.py           # FastAPI routes, engine registry, dynamic-prefix decoding
│   ├── schemas.py        # ChatRequest / ChatResponse / HealthResponse (pydantic)
│   ├── inference.py       # Checkpoint loader + raw autoregressive generation
│   └── static/
│       └── index.html     # Chat UI (model/tier picker, temperature, max_tokens)
├── src/nanogpt/
│   ├── config.py          # ModelConfig / TrainConfig / SFTConfig dataclasses
│   ├── tokenizers.py      # `train` / `encode` subcommands for the BPE tokenizer
│   ├── dataset.py         # Memmap pretrain dataset, SFT dataset, overfit dataset
│   ├── model.py           # GPT: causal attention (SDPA) + MLP blocks, weight tying
│   ├── train_base.py      # Base pre-training loop (always reads configs/base.yaml)
│   ├── judge.py           # Ollama/Qwen 2.5: single-question SFT pairs
│   ├── judge1.py          # Ollama/Qwen 2.5: multi-question SFT pairs
│   ├── merge_data.py      # Merges judge.py + judge1.py output (hardcoded paths)
│   ├── train_sft.py        # Answer-only masked-loss fine-tuning
│   ├── judge_dpo.py       # Candidate generation + Qwen chosen/rejected grading
│   ├── train_dpo.py        # Custom DPO trainer (CLI-driven, no YAML)
│   ├── generate_targeted_dpo.py # Mines chronology/list-extraction failures → v2 data
│   └── evaluate.py         # Validation loss, perplexity, latency benchmarking
├── configs/
│   ├── base.yaml          # Architecture + pre-training hyperparameters (Fire, by default)
│   ├── sft_15M.yaml       # SFT hyperparameters – Fiko
│   └── sft_33M.yaml       # SFT hyperparameters – Fire
├── scripts/
│   ├── download_data.py    # Pulls TinyStories + TinyStories-Instruct via 🤗 `datasets`
│   └── overfit_test.py     # Small-batch overfit sanity check
├── checkpoints/            # base_model_33M.pt, sft_model_33M.pt, dpo_model_33M_v1/v2.pt
│                           # - tracked with Git LFS; *.pt is intentionally NOT in .gitignore
├── tests/
│   └── test_model.py       # Shape, weight-tying, causality, ignore_index tests
├── sample.py              # Standalone unconditional generation from the base checkpoint
├── test_cot.py            # Standalone Answer-Prefix-Priming demo script
├── tokenizer.json
├── Dockerfile             # python:3.11-slim, installs requirements.txt, serves on :7860
├── Makefile               # setup / tokenize / train / judge / sft / serve / test
├── pyproject.toml
├── requirements.txt        # Production/serving deps – CPU-only torch
└── README.md

```

`data/` is git-ignored entirely; `checkpoints/` deliberately is **not** (see the comment in `.gitignore`: “*Model checkpoints (See deployment note below!)*”) — the deployed container needs the real `.pt` files in the image, so they’re committed via Git LFS instead of being excluded like a typical ML repo would exclude them.

Part II — Training Guide

All commands below are the actual CLI accepted by the scripts in this repo (verified against their `argparse` definitions), not idealized examples — including a few rough edges called out inline so you don't lose time to them.

Contents

- Setup
- Configuration Reference
- Phase 1: Download Data
- Phase 2: Tokenizer
- Phase 3: Base Pre-training
- Phase 4: Hybrid SFT Data Generation
- Phase 5: Supervised Fine-Tuning (Medium tier)
- Phase 6: DPO Data Generation
- Phase 7: DPO Alignment (High & Max tiers)
- Phase 8: Evaluation
- Phase 9: Ad-hoc Generation Scripts
- Makefile Shortcuts
- Direct Preference Optimization, In Detail
- Prompt Format

Setup

Prerequisites

- Python 3.10+
- **Git LFS** (`git lfs install`) — the `checkpoints/*.pt` files are LFS objects; without it you'll only get small pointer files
- An NVIDIA GPU with Ampere/Ada Lovelace architecture for bfloat16 (e.g. RTX 3050+, RTX 4050+) — training only; serving runs fine on CPU
- [Ollama](#) installed and on your system path, for the data-generation phases

Install

```
git lfs install
git clone https://github.com/yogeshsikhwal77/nanoGPT.git
cd nanoGPT

python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1

# CUDA build of torch for training (the repo's own requirements.txt installs CPU-only torch,
# which is what the deployed service uses — swap it out for training):
pip install torch --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt
pip install -e .

# Not declared in requirements.txt / pyproject.toml but required for data generation:
pip install datasets ollama

ollama pull qwen2.5:3b
```

Verify GPU visibility:

```
python -c "import torch; print('CUDA Available:', torch.cuda.is_available(), '| Device:',
torch.cuda.get_device_name(0), '| BF16 Supported:', torch.cuda.is_bf16_supported())"
```

Note: `pyproject.toml` declares `torch`, `tokenizers`, `pyyaml`, `fastapi`, `uvicorn`, `tqdm` as dependencies, and the root `requirements.txt` (used by `Dockerfile`, i.e. production) installs `numpy`, `fastapi`, `uvicorn`, `tokenizers`, `pydantic` against the **CPU** PyTorch index. Neither file lists `datasets` or `ollama` (the Python client), even though `scripts/download_data.py`, `judge.py`, `judge1.py`, `judge_dpo.py`, and `generate_targeted_dpo.py` all import them — install them manually as shown above.

Configuration Reference

`configs/base.yaml` — read unconditionally by `train_base.py`

```
model:
  dim: 512          # widened residual stream (was 384)
  layers: 8         # deeper reasoning capacity (was 6)
  heads: 8          # 512 / 8 = 64 head dim
  context_len: 512  # can handle longer, richer stories (was 256)
  vocab_size: 10000

train:
  batch_size: 16
  grad_accum: 4     # effective batch = 64
  lr: 8.0e-4
  epochs: 4         # 4 passes over the data (~50-60 min on an RTX 4050)
  seed: 42
  data_dir: data/tokenized/
  save_path: checkpoints/base_model_33M.pt
```

Important: `train_base.py` calls `load_base_config("configs/base.yaml")` with a hardcoded path — it does not accept a `--config` flag (the Makefile's `train` target passes one, but the script silently ignores it). To train Fiko (15M), you must edit `configs/base.yaml` in place (or back it up, swap in a smaller-dims version, train, then restore it) — there is no `base_15M.yaml` shipped in this repo. A Fiko-sized `model:` block matching the numbers in Part I would be `dim: 384, layers: 6, heads: 6, context_len: 256`, with `save_path: checkpoints/base_model_15M.pt`.

`configs/sft_33M.yaml`

```
model_path: "checkpoints/base_model_33M.pt"
data_path: "data/sft/merged_pairs.json"
save_path: "checkpoints/sft_model_33M.pt"
batch_size: 8
grad_accum: 2      # effective batch = 16
lr: 1.5e-4         # slightly gentler for fine-tuning
warmup_steps: 30   # ~10% of total steps
epochs: 2          # 2 epochs is the sweet spot to prevent memorization
min_lr_ratio: 0.1
```

configs/sft_15M.yaml

```

model_path: checkpoints/base_model_15M.pt
data_path: data/sft/aligned_pairs_1000.json # note: NOT merged_pairs.json – see caveat below
save_path: checkpoints/sft_model_15M.pt
batch_size: 8
grad_accum: 2
lr: 2.0e-4
epochs: 2
warmup_steps: 50

```

Unlike `sft_33M.yaml`, the shipped `sft_15M.yaml` points at `data/sft/aligned_pairs_1000.json` (the multi-question set alone) rather than the merged single+multi dataset — repoint it at `data/sft/merged_pairs.json` if you want Fiko trained on the same hybrid data as Fire.

DPO parameters — CLI flags, not a YAML file

`train_dpo.py` takes all of its hyperparameters as CLI flags (`--epochs`, `--batch-size`, `--grad-accum`, `--lr`, `--beta`); there is no `dpo.yaml` in this repo. See Phase 7 below for the exact values used in the recorded run.

Phase 1: Download Data

```
python scripts/download_data.py
```

Downloads the first 300,000 rows of `roneneldan/TinyStories` to `data/raw/train.txt` (pre-training corpus), and `roneneldan/TinyStories-Instruct` to `data/raw/TinyStories-Instruct.json` (seed material for the SFT judges). Requires the `datasets` package.

Phase 2: Tokenizer

```

# Train the 10k-vocab byte-level BPE tokenizer
python -m nanogpt.tokenizers train --vocab-size 10000 --data-path data/raw

# Encode the raw text into memory-mapped uint16 shards
python -m nanogpt.tokenizers encode --input-dir data/raw/ --output-dir data/tokenized/

```

Produces `tokenizer.json` and `data/tokenized/shard_0000.bin`.

Phase 3: Base Pre-training

```

# Fire (33M) – configs/base.yaml as shipped
python -m nanogpt.train_base

# Fiko (15M) – edit configs/base.yaml first (dim:384, layers:6, heads:6, context_len:256,
# save_path: checkpoints/base_model_15M.pt), then run the same command
python -m nanogpt.train_base

```

Trains with a cosine LR schedule, checkpoints only on a new best validation loss, and prints elapsed time every 50 steps.

Phase 4: Hybrid SFT Data Generation

```
# 1. Single-question pairs
python -m nanogpt.judge --instruct-data data/raw/TinyStories-Instruct.json --output data/sft/aligned_pairs.json --max-samples 500

# 2. Multi-question pairs — MUST override --output, since judge1.py's default
#   ("data/sft/aligned_pairs.json") is the same as judge.py's and would overwrite it
python -m nanogpt.judge1 --instruct-data data/raw/TinyStories-Instruct.json --output data/sft/aligned_pairs_1000.json --max-samples 500

# 3. Merge — no CLI args; hardcoded to read exactly
#   data/sft/aligned_pairs.json + data/sft/aligned_pairs_1000.json
python -m nanogpt.merge_data
```

`judge.py` and `judge1.py` also accept `--model-path` and `--candidates` flags, but both are explicitly ignored by the current implementation (`help="Ignored"` in their own `argparse` definitions) — don't rely on them. Output: `data/sft/merged_pairs.json`.

Phase 5: Supervised Fine-Tuning (Medium tier)

```
# Fire (33M)
python -m nanogpt.train_sft --config configs/sft_33M.yaml

# Fiko (15M)
python -m nanogpt.train_sft --config configs/sft_15M.yaml
```

Optional flags: `--val-fraction` (default `0.05`) and `--eval-every` (optimizer steps between validation passes, default `100`). Loss is computed only on answer tokens (masked with `-100` on the story/question span).

Phase 6: DPO Data Generation

```
# Generate candidate answers from the SFT checkpoint and have Qwen grade chosen/rejected
python -m nanogpt.judge_dpo \
  --sft-checkpoint checkpoints/sft_model_33M.pt \
  --seed-data data/sft/aligned_pairs.json \
  --output data/dpo/dpo_pairs.json \
  --max-pairs 500
```

This is the pass used for baseline DPO (v1). The targeted, harder dataset for v2 is mined **after** v1 exists — see the next phase.

Phase 7: DPO Alignment (High & Max tiers)

```
# Fire — High tier: baseline DPO (v1)
python -m nanogpt.train_dpo \
  --sft-checkpoint checkpoints/sft_model_33M.pt \
  --dpo-data data/dpo/dpo_pairs.json \
  --save-path checkpoints/dpo_model_33M_v1.pt \
  --epochs 1 --batch-size 2 --grad-accum 4 --lr 2e-6 --beta 0.1

# Mine targeted chronology/list-extraction failures from the v1 checkpoint
# (override --model-path — its default, checkpoints/dpo_model_33M.pt, doesn't match any
# file this pipeline actually produces)
python -m nanogpt.generate_targeted_dpo \
  --model-path checkpoints/dpo_model_33M_v1.pt \
  --seed-data data/sft/aligned_pairs.json \
  --output data/dpo/targeted_dpo_pairs.json \
  --max-stories 100

# Fire — Max tier: targeted DPO (v2), gentler lr/beta than v1
python -m nanogpt.train_dpo \
  --sft-checkpoint checkpoints/dpo_model_33M_v1.pt \
  --dpo-data data/dpo/targeted_dpo_pairs.json \
  --save-path checkpoints/dpo_model_33M_v2.pt \
  --epochs 1 --batch-size 2 --grad-accum 4 --lr 1e-6 --beta 0.08
```

Repeat the same three commands with `_15M` in place of `_33M` (and `sft_model_15M.pt` as the SFT checkpoint) to produce Fiko's High and Max checkpoints — `checkpoints/dpo_model_15M_v2.pt` is required for the `fiko-max` engine to load in `app/main.py` (see Part III).

Phase 8: Evaluation

```
python -m nanogpt.evaluate --checkpoint checkpoints/dpo_model_33M_v2.pt \
  --data-dir data/tokenized --tokenizer-path tokenizer.json \
  --batch-size 16 --iters 50
```

Reports validation loss, perplexity, and generation latency.

Phase 9: Ad-hoc Generation Scripts

Two standalone (non-`-m`) scripts exist for quick manual checks:

```
python sample.py      # unconditional generation from checkpoints/base_model_33M.pt
python test_cot.py    # Answer-Prefix-Priming demo — story is hardcoded in the script
```

Heads up: `test_cot.py` hardcodes `checkpoints/dpo_model_33M.pt` (no `_v1` / `_v2` suffix), which this pipeline never actually produces — copy/rename a checkpoint to that exact filename, or edit the path in the script, before running it.

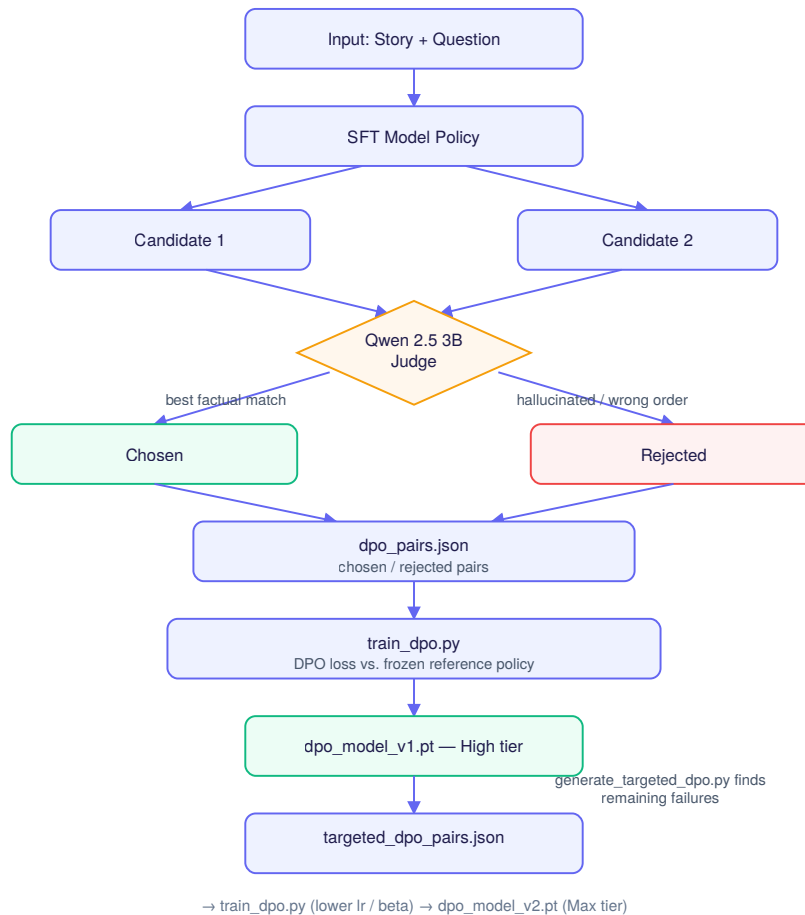
`test_cot.py`'s "Answer Prefix Priming" idea — steering the model's attention past common logic traps by priming the start of the answer span — has since been generalized and moved into the live server itself: `app/main.py`'s `get_dynamic_prefix()` applies equivalent heuristics (for "what two...", "why...", "when...", "where did X go", "what did X do after/before Y...") to every `/chat` request, not just this offline demo. See Part III.

Makefile Shortcuts

```
make setup      # CUDA torch + requirements.txt + editable install + `ollama pull llama3.2:1b`
make tokenize   # python -m nanogpt.tokenizers encode --input-dir data/raw/ --output-dir data/tokenized/
make train      # python -m nanogpt.train_base --config configs/base.yaml (the --config flag is ignored, see above)
make judge      # python -m nanogpt.judge --model-path checkpoints/base_model_15M.pt \
                # --instruct-data data/raw/TinyStories-Instruct.json \
                # --output data/sft/aligned_pairs.json --candidates 3
make sft        # python -m nanogpt.train_sft --config configs/sft.yaml (configs/sft.yaml doesn't exist — pass configs/
sft_33M.yaml or sft_15M.yaml explicitly instead)
make serve      # uvicorn app.main:app --host 127.0.0.1 --port 8000
make test       # pytest -q
```

Two things to know before relying on `make` : `make setup` pulls `llama3.2:1b` via Ollama, but every judge/DPO script in `src/nanogpt/` is hardcoded to call `qwen2.5:3b` — pull that model instead (`ollama pull qwen2.5:3b`). And `make sft` points at `configs/sft.yaml` , which isn't in the repo — use `make` 's command as a template but pass `--config configs/sft_33M.yaml` (or `sft_15M.yaml`) yourself. There are no Makefile targets for the DPO phases; use the `python -m nanogpt.judge_dpo / train_dpo / generate_targeted_dpo` commands above directly.

Direct Preference Optimization, In Detail



Why DPO instead of RLHF with PPO? DPO reformulates preference alignment as a single classification-style loss between a policy and a frozen reference model, using only chosen/rejected pairs — no separate reward model or on-policy rollouts to stabilize. That's

dramatically cheaper to implement and run, which matters when the whole loop needs to fit inside 6 GB of VRAM. `train_dpo.py` loads the same checkpoint twice — once as the trainable policy, once frozen (`requires_grad=False`) as the reference — so the added memory cost over plain SFT is one extra forward pass, not a second trainable model.

Why a targeted v2 set? After the first DPO pass, the most persistent errors cluster around a few specific patterns — chronological ordering ("what happened first?") and multi-item list extraction ("what two things did she bring?"). `generate_targeted_dpo.py` searches the v1 checkpoint's outputs for these failure modes and builds a second, harder preference dataset focused on fixing them, trained with a lower learning rate and beta for a gentler correction (see the recorded run in Part I, where the v2 pass's loss moved measurably more than v1's did).

Prompt Format

SFT and inference both use strict special-token boundaries:

```
<|story|> {story} <|question|> {question} <|answer|> {answer} <|eos|>
```

DPO preference pairs share the same story/question prefix with two candidate continuations:

```
Prompt:  <|story|> {story} <|question|> {question} <|answer|>
Chosen:  {factually correct answer} <|eos|>
Rejected: {hallucinated or order-confused answer} <|eos|>
```

At inference, input stops at `<|answer|>` and generation samples tokens autoregressively until `<|eos|>` or `max_tokens` is reached. Inputs longer than the model's context window are truncated **from the start of the story**, preserving the entire question.

Part III — Serving & Reference

Contents

- Deployment Notes
- API Reference
- Hardware and Memory Budget
- Troubleshooting
- Limitations
- Roadmap
- License

Deployment Notes

The live demo (nanogpt-ic7p.onrender.com) runs the same `app/main.py` FastAPI app you'd run locally, built from the repo's `Dockerfile`:

```
FROM python:3.11-slim
WORKDIR /code
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# Hugging Face Spaces routes traffic to port 7860 by default
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "7860"]
```

Two things worth knowing:

- The port comment references **Hugging Face Spaces** conventions, but the current live deployment is on **Render** — the image itself is portable to either.
- `requirements.txt` installs PyTorch from the CPU wheel index (`--extra-index-url https://download.pytorch.org/whl/cpu`), so the production container does inference on CPU, not GPU.

How model selection actually works

`app/schemas.py`'s `ChatRequest.model_name` accepts six values: `fire-medium`, `fire-high`, `fire-max`, `fiko-medium`, `fiko-high`, `fiko-max` — and the chat UI (`app/static/index.html`) presents all six as separate dropdown options. In production, though, `app/main.py`'s startup routine only ever loads **two actual model instances**:

```
models_to_load = {
    "fire-max": "checkpoints/dpo_model_33M_v2.pt",
    "fiko-max": "checkpoints/dpo_model_15M_v2.pt",
}
# each family's "medium" and "high" selections are aliased to that family's "max" engine:
engines["fire"] = engines["fire-medium"] = engines["fire-high"] = engines.get("fire-max")
engines["fiko"] = engines["fiko-medium"] = engines["fiko-high"] = engines.get("fiko-max")
```

This is a deliberate memory-saving measure — the comment in the source calls it out explicitly: *"Only load the 'Max' models to stay under Render's 512MB Free Tier limit."* Practically, this means:

- Selecting `fire-medium` or `fire-high` in the UI does **not** serve the actual SFT or DPO-v1 checkpoint — it silently serves `fire-max`'s output instead.

- Same for `fiko-medium` / `fiko-high` → `fiko-max`.
- Each engine only loads `if os.path.exists(path)` — as of the current commit, `checkpoints/` in this repository contains only the Fire (33M) family (`base_model_33M.pt` , `sft_model_33M.pt` , `dpo_model_33M_v1.pt` , `dpo_model_33M_v2.pt`), tracked via Git LFS. There is no `dpo_model_15M_v2.pt` in the tree, so `fiko-max` (and therefore every `fiko-*` selection) has no engine to route to, and any `/chat` call with a `fiko-*` model name returns `404 Model 'fiko-max' is not loaded`. Whether the live Render deployment has a Fiko checkpoint added outside of this repo isn't something this documentation can confirm from the source alone — if you want Fiko to actually work (locally or in production), train it per Part II and commit `checkpoints/dpo_model_15M_v2.pt` via Git LFS.

Generation behavior in production

Every `/chat` request goes through `app/main.py`'s `generate_guided()`, which is a superset of the "Answer Prefix Priming" idea from `test_cot.py`: `get_dynamic_prefix()` inspects the question text and, for several question patterns (`what two...` / `name two...` list queries, `why...` , `when...` , `where did X go` , `what did X do after/before Y`), primes the model's answer with a short natural-language prefix before decoding — with a fallback to unprimed decoding if the primed attempt produces nothing. List-style questions also force at least 6 tokens of generation before EOS is allowed, so the model can't stop after naming only one of two requested items.

API Reference

Schema is defined in `app/schemas.py` — the fields below are exactly what's implemented (no more, no less).

POST /chat

```
curl -X POST http://127.0.0.1:8000/chat \
  -H "Content-Type: application/json" \
  -d '{
    "story": "Tim lost his red ball in the garden. He searched behind the shed and found it under the oak tree.",
    "question": "Where did Tim find his ball?",
    "temperature": 0.2,
    "max_tokens": 60,
    "model_name": "fire-max"
  }'
```

Request fields

Field	Type	Default	Notes
<code>story</code>	string	required	Truncated from the start if it exceeds the model's context window
<code>question</code>	string	required	
<code>temperature</code>	float	<code>0.2</code>	Range <code>0.0</code> – <code>2.0</code> ; <code>0</code> means greedy (argmax) decoding
<code>max_tokens</code>	int	<code>50</code>	Range <code>1</code> – <code>128</code>
<code>model_name</code>	string	<code>"fire-max"</code>	One of <code>fire-medium</code> , <code>fire-high</code> , <code>fire-max</code> , <code>fiko-medium</code> , <code>fiko-high</code> , <code>fiko-max</code> — see Deployment Notes for which actually differ

Response

```
{
  "answer": "Under the oak tree.",
  "tokens_generated": 6,
  "inference_time_ms": 27.4
}
```

`ChatResponse` has exactly these three fields — there's no `model_stage` field in the implementation.

Errors

Status	When
422	Missing/invalid field (standard FastAPI/pydantic validation)
404	<code>model_name</code> isn't a key in the loaded <code>engines</code> dict — the response body includes the list of currently-available engine names

GET /health

```
{
  "status": "ok",
  "device": "cpu",
  "vram_used_mb": 0.0
}
```

`HealthResponse` has exactly these three fields — there's no `loaded_checkpoint` field in the implementation. Returns

503 No models loaded yet if the `engines` dict is empty (e.g. neither checkpoint file was found at startup).

GET /

Serves `app/static/index.html` (the chat UI) via `FileResponse`; returns 404 if that file is missing.

CORS is wide open (`allow_origins=["*"]`) on every route.

Hardware and Memory Budget

Training is engineered to fit inside a 6 GB VRAM budget, across every phase including DPO:

- **bfloat16 autocast** — cuts activation memory roughly in half vs. float32, without the underflow/overflow issues of full fp16.
- **FlashAttention (SDPA)** — `F.scaled_dot_product_attention` replaces materializing the $O(T^2)$ attention matrix in VRAM with tile-based fused kernels.
- **Memory-mapped pre-tokenization** — the training dataset is accessed directly from disk via `np.memmap(dtype=np.uint16)`; training uses well under 300 MB of system RAM regardless of dataset size.
- **Tied embeddings** — sharing weights between `tok_emb` and `lm_head` saves parameters and checkpoint size.
- **Frozen reference policy for DPO** — the DPO reference model runs inference-only (no gradients), so the extra memory cost over standard SFT is one extra forward pass, not a second trainable copy.
- **Local judge instead of a hosted API** — Qwen 2.5 3B runs through Ollama on the same machine, so data generation and judging never leave your GPU/CPU.

If you hit CUDA out-of-memory during training, lower `batch_size` and raise `grad_accum` to keep the effective batch size constant.

Serving runs on CPU by design (see Deployment Notes) and only two ~33M/15M-parameter checkpoints are ever resident in memory at once — the explicit reason the app only loads the Max tier of each family on Render's free tier.

Troubleshooting

Issue	Root cause	Solution
Checkpoints are ~130 bytes of text, not real weights	Cloned without Git LFS	Run <code>git lfs install</code> before cloning, or <code>git lfs pull</code> in an existing clone
<code>train_base.py</code> trains the same architecture no matter what <code>--config</code> you pass	The script hardcodes <code>load_base_config("configs/base.yaml")</code> and ignores CLI args entirely	Edit <code>configs/base.yaml</code> directly before running, or swap files in place
<code>judge1.py</code> 's output overwrites <code>judge.py</code> 's output	Both scripts default <code>--output</code> to <code>data/sft/aligned_pairs.json</code>	Always pass an explicit, different <code>--output</code> to <code>judge1.py</code> (e.g. <code>data/sft/aligned_pairs_1000.json</code>)
<code>merge_data.py</code> can't find your files	It has no CLI args — hardcoded to read <code>data/sft/aligned_pairs.json</code> and <code>data/sft/aligned_pairs_1000.json</code>	Name your judge outputs to match, or edit the two paths in <code>merge_data.py</code>
<code>generate_targeted_dpo.py --model-path</code> default doesn't exist	Its default is <code>checkpoints/dpo_model_33M.pt</code> , but the pipeline only ever produces <code>..._v1.pt</code> / <code>..._v2.pt</code>	Always pass <code>--model-path</code> explicitly
<code>test_cot.py</code> fails to find its checkpoint	Hardcodes <code>checkpoints/dpo_model_33M.pt</code> (no version suffix)	Copy/rename a checkpoint to that filename, or edit the path in the script
<code>ModuleNotFoundError: No module named 'datasets' / 'ollama'</code>	Not declared in <code>requirements.txt</code> or <code>pyproject.toml</code>	<code>pip install datasets ollama</code>
<code>ollama pull llama3.2:1b</code> (from <code>make setup</code>) but judge scripts fail to find a model	The Makefile pulls a different model than the judge/ DPO scripts actually call (<code>qwen2.5:3b</code> , hardcoded in <code>judge.py</code> / <code>judge1.py</code> / <code>judge_dpo.py</code> / <code>generate_targeted_dpo.py</code>)	<code>ollama pull qwen2.5:3b</code>
<code>RuntimeError: Error(s) in loading state_dict for GPT</code>	Architecture mismatch between the config used to load a checkpoint and the one it was trained with	Verify <code>dim</code> , <code>layers</code> , <code>heads</code> , and <code>context_len</code> match the values used to train the checkpoint
DPO loss stays near $\ln(2) \approx 0.693$ for the whole run	Expected for a v1/baseline pass on a small, low-signal preference set — see the recorded run in Part I	If it never moves even slightly, check <code>beta</code> isn't set too high, and confirm the reference model was loaded with <code>requires_grad=False</code>
Qwen judge returns inconsistent verdicts, or is biased toward one candidate slot	Non-zero temperature on judge calls, or positional bias in the judge prompt	Use temperature <code>0</code> ; if one candidate slot (e.g. "B") wins suspiciously often, spot-check a sample manually — see the note in Part I
<code>torch.cuda.is_available()</code> is <code>False</code> during training	PyTorch installed without CUDA runtime dependencies, or you're using the CPU wheel from <code>requirements.txt</code>	Reinstall with CUDA wheels: <code>pip install torch --index-url https://download.pytorch.org/whl/cu121</code>
ByteLevel special characters (<code>Ġ</code> , <code>Ĉ</code>) in generated text	Tokenizer missing the ByteLevel decoding pipeline	Set <code>tokenizer.decoder = ByteLevel()</code> before calling <code>.decode()</code> (already done in <code>app/inference.py</code> , but easy to miss in a standalone script)
<code>fiko-*</code> requests 404 on <code>/chat</code>	<code>checkpoints/dpo_model_15M_v2.pt</code> isn't present	See Deployment Notes

Limitations

- Domain-bound to simple children's stories; no broad world knowledge.
- Weak at multi-step arithmetic even after DPO; alignment mainly corrects extraction/ordering errors, not reasoning depth.
- Context window (256 tokens for Fiko, 512 for Fire) still truncates longer stories.
- SFT and DPO data quality depend on a 3B local judge, smaller and noisier than frontier judge models — spot-checking accepted pairs is recommended, especially given the candidate-ordering bias noted in Part I.
- "Medium" and "High" tiers are not actually served in production for either model family — only "Max" is live (see Deployment Notes).
- Fiko has no committed checkpoints in this repo as of now, so it isn't runnable out of the box.
- Much smaller than modern general-purpose LLMs; intended for learning and experimentation, not production use.

Roadmap

- [] Commit (or otherwise make available) `checkpoints/dpo_model_15M_v2.pt` so `fiko-max` actually loads
- [] Either wire up real Medium/High engines per family, or drop those options from the UI to match what's actually served
- [] Give `train_base.py` a `--config` flag instead of a hardcoded path, and add a `base_15M.yaml`
- [] Add a `configs/dpo.yaml` so DPO hyperparameters don't have to be re-typed on the command line every run
- [] Fix the default paths in `judge1.py` and `generate_targeted_dpo.py` so they don't collide with / miss real outputs
- [] Publish full DPO win-rate ablations (base vs. SFT vs. DPO v1 vs. DPO v2) for both Fiko and Fire
- [] Add tokenizer round-trip and SFT-masking tests alongside `test_model.py`
- [] KV-cache for faster generation
- [] Streaming responses in the chat UI
- [] Swap in a larger local judge (Qwen 2.5 7B) as an ablation, and check whether it reduces the candidate-ordering bias observed with the 3B judge

License

MIT — see `LICENSE`. Copyright (c) 2026 Yogesh Sikhwal.

ABOUT THIS DOCUMENT

This reference merges three source documents — *Architecture & Pipeline*, *Training Guide*, and *Serving & Reference* — into a single, continuously-paginated technical manual for the nanoGPT project. Diagrams have been redrawn for consistent typography and print layout; all commands, configuration values, and API contracts are reproduced exactly as documented in the source material, with cross-references between parts updated to point within this document rather than across separate files.